

Actividad de Laboratorio: Arquitectura Multi-Nivel (N-Tier) y Patrón Lógico de Software (MVC) en .NET

Duración estimada: 120 minutos (2 horas)

Modalidad: Individual

Entregables requeridos: 1. Archivo Reporte.md con la investigación teórica, análisis de enrutamiento y referencias bibliográficas estructuradas.

2. Directorio con el código fuente del proyecto web funcional en C#.

Parte 1: Fundamentación Teórica y Análisis Crítico

El objetivo de esta fase es asimilar las diferencias entre la distribución física del hardware y la organización interna del código, identificando por qué las arquitecturas modernas prohíben el acoplamiento de componentes.

1. El Tránsito hacia los Sistemas Distribuidos y Multi-Capa

- **La Limitación del Monolito Local:** Explica qué ocurre con la sincronización y escalabilidad de los datos cuando la interfaz, la lógica y el almacenamiento residen de forma exclusiva en una máquina física aislada .
- **Distinción Crítica (Layers vs. Tiers):** Define con precisión la diferencia conceptual entre las *Capas Lógicas* y los *Niveles Físicos* .
- **Responsabilidades en la Arquitectura de 3 Niveles:** Describe brevemente cuál es la misión exclusiva y la tecnología común empleada en cada uno de los siguientes niveles físicos:
 - Nivel 1: Capa de Presentación (*Presentation Tier*) .
 - Nivel 2: Capa de Aplicación o Negocio (*Application Tier*) .
 - Nivel 3: Capa de Datos (*Data Tier*) .
- **Seguridad Perimetral:** Justifica desde el punto de vista de la ingeniería por qué es un error crítico de seguridad exponer públicamente el puerto de una base de datos a internet y cuál es la buena práctica recomendada para su protección .

2. Desacoplamiento Lógico con el Patrón MVC

- **La Crisis del Código Espagueti:** Describe qué impactos negativos tiene sobre el mantenimiento del software y el diseño de pruebas unitarias el hecho de mezclar sentencias SQL, lógica matemática y etiquetas visuales dentro de un mismo archivo físico .
- **Separación de Preocupaciones (SoC):** Detalle las características esenciales y el nivel de aislamiento de los tres componentes del patrón formulado por Trygve Reenskaug :
 - **Modelo:** Qué representa y por qué no debe conocer cómo se muestran los datos .

- **Vista:** Por qué se define como una entidad *pasiva e inteligente* y qué tipo de código tiene estrictamente prohibido contener .
- **Controlador:**Cuál es su rol como intermediario táctico y director de orquesta en la red .
- **Métricas de Ingeniería de Software:** Explica cómo el patrón MVC ayuda a alcanzar una **Alta Cohesión** y un **Bajo Acoplamiento** en un entorno de desarrollo profesional .

Parte 2: Modelado del Ciclo de Vida y Enrutamiento Semántico

1. Mapeo Analítico de URLs

El motor de enrutamiento convencional de ASP.NET Core utiliza por defecto la plantilla jerárquica:

`\{controller=Home\}/\{action=Index\}/\{id?\}`

Con base en esto y el ejemplo matemático de la Sesión 12 , completa la siguiente tabla de mapeo de rutas semánticas :

URL Entrante del Cliente	Clase Controladora Buscada por el Framework	Método (Acción) Ejecutado	Parámetro id Inyectado
https://ingenieria.usac.edu.gt/ControlAcademico/Login			
https://ingenieria.usac.edu.gt/Estudiante/Historial/20260123	EstudianteControlador	Historial	20260123
https://ingenieria.usac.edu.gt/Asignacion/Detalle/10			
https://ingenieria.usac.edu.gt/Home			(Ninguno Opcional) /

2. Diagramación del Flujo Interactivo

Redacta de forma secuencial (paso a paso, del 1 al 5) el viaje completo que realiza una petición web HTTP desde que un usuario hace clic en un botón en su navegador web hasta que recibe el código HTML renderizado dinámicamente, detallando cuándo interviene cada componente del patrón MVC .

Parte 3: Implementación Práctica - Sistema de Control Académico

Para consolidar la teoría, construirás una aplicación interactiva utilizando controladores nativos bajo **ASP.NET Core (MVC)** en .NET 8, emulando la gestión de estudiantes y aplicando de forma estricta las buenas prácticas de diseño de la industria.

Requisitos Técnicos:

- Separación de responsabilidades en archivos independientes .
- Implementación de controladores delgados (*Skinny Controllers*) que no superen las 20 líneas de código por método .

Instrucciones paso a paso:

1. Creación del espacio de trabajo:

Inicializa un proyecto web estructurado con soporte para controladores:

```
dotnet new webapp -o ControlAcademicoMvc  
cd ControlAcademicoMvc
```

2. Estructuración del Modelo (Capa de Dominio): Crea una carpeta llamada Models/ y dentro de ella define la entidad pura de C# (POCO) para representar a un estudiante :

```
// Archivo: Models/Estudiante.cs  
public class Estudiante  
{  
    public int Carne { get; set; }  
    public string Nombre { get; set; } = string.Empty;  
    public double Promedio { get; set; }  
}
```

3. Implementación del Controlador (Despachador Perimetral): Crea una carpeta llamada Controllers/ y añade la clase controladora. Aplica la regla de oro: el controlador solo redirige el tráfico y delega las operaciones.

```
// Archivo: Controllers/EstudianteController.cs  
using Microsoft.AspNetCore.Mvc;
```

```

using ControlAcademicoMvc.Models;

public class EstudianteController : Controller
{
    // Almacenamiento simulado en memoria centralizada (Simulando Tier 3)
    private static readonly List<Estudiante> _baseDatosMemoria = new()
    {
        new Estudiante { Carne = 2026012, Nombre = "Fernando Velásquez", Promedio = 91.5 },
        new Estudiante { Carne = 2026045, Nombre = "María Mercedes", Promedio = 84.0 }
    };

    // GET: /Estudiante/Listar
    public IActionResult Listar
    {
        // El controlador extrae los datos limpios del modelo y los inyecta a la vista
        return View(_baseDatosMemoria);
    }

    // POST: /Estudiante/Registrar
    [HttpPost]
    public IActionResult Registrar
    {
        // Validación perimetral rápida (Skinny Controller)
        if (nuevoEstudiante.Carne <= 0 || string.IsNullOrEmpty(nuevoEstudiante.Nombre))
        {
            return BadRequest(new { mensaje = "Datos del estudiante inválidos." });
        }

        _baseDatosMemoria.Add(nuevoEstudiante);
        return Created($"/Estudiante/Historial/{nuevoEstudiante.Carne}", nuevoEstudiante);
    }
}

```

4. **Habilitación del Enrutamiento por Controladores en Program.cs:** Asegúrate de que tu archivo Program.cs tenga activado el soporte para controladores convencionales mapeando la plantilla jerárquica de la unidad:

```

var builder = WebApplication.CreateBuilder(args);

// Agregar servicios al contenedor de dependencias
builder.Services.AddControllersWithViews();

var app = builder.Build();

```

```
app.UseStaticFiles();
app.UseRouting();

// Configuración de la plantilla estándar de enrutamiento por convención
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

Parte 4: Auditoría y Control de Calidad

Una vez completado el código, ejecuta el proyecto (dotnet run) y realiza una verificación interna utilizando Postman o tu navegador para validar la calidad del desacoplamiento:

1. **Prueba de Cohesión (GET):** Accede a la acción de listado. ¿La respuesta es limpia y el controlador se limitó a despachar la información sin calcular variables internas o mezclar sentencias SQL en texto plano? .
2. **Evaluación de Antipatrones:** Revisa el archivo EstudianteController.cs. Si alguno de tus métodos excede las 20 líneas de código, realiza una refactorización inmediata para evitar el antipatrón de *Controladores Gordos (Fat Controllers)* .

Parte 5: Referencias Bibliográficas (Requisito Formal de Entrega)

Para que la actividad sea evaluada y dada por válida por el cuerpo docente, el estudiante debe listar las fuentes oficiales consultadas en la base de su reporte Markdown, utilizando las referencias normativas del laboratorio:

- **Referencia Central 1:** > Facultad de Ingeniería, USAC. (2026). *Sesión 11: Modelado Base y Arquitecturas de Despliegue. Evolución de Sistemas Distribuidos, Fundamentos del Modelo Cliente-Servidor y Diseño Físico Multi-Capas (N-Tier)*. Laboratorio del curso Introducción a la Programación y Computación 2. Guatemala.
- **Referencia Central 2:** > Facultad de Ingeniería, USAC. (2026). *Sesión 12: Arquitectura y Componentes del Patrón MVC. Desacoplamiento Lógico de Software, Ciclo de Vida de las Peticiones y Enrutamiento en Aplicaciones Interactivas Modernas*. Laboratorio del curso Introducción a la Programación y Computación 2. Guatemala.